DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA06 — DESIGN AND ANALYSIS OF ALGORITHMS

Continuous Assessment Assessment (CO2): Debugging and Optimisation Task

SOC Alert-Triage Module — SentinelOps Incident Response Platform

Student Name:	R.VISSHAAL	**Registration Number:**	192421025
Course Code & Name:	CSA06 — Design & Analysis of Algorithms	**Submission Date:**	26 October 2026
Assessment Component:	CO2 Debugging & Optimisation	**Maximum Marks:**	50 Marks
GitHub Repository:	https://github.com/visshaalpvt/CO2-Assessment-Tool-2-Debugging-and-Optimization-Task-SOC-Alert-Triage-Module-of-SentinelOps.git	**Target Branch:**	master / main

1. Problem Identification

The SentinelOps Security Operations Centre (SOC) alert-triage module processes critical incident data: prioritising security alerts by severity (stable descending), filtering blocked IP addresses via blocklist lookup, scanning raw network audit logs for cyber-attack signatures, and detecting concurrent anomalous logins via geographical closest-pair analysis. The original module supplied in original_buggy.py contained nine distinct defects that caused alert corruption, security bypasses, missed attack detections, and selection of maximum distances instead of closest pairs.

1.1 Complete Nine-Defect Register

#	Function	Line	Faulty Statement	Root Cause & Defect Mechanism	Actual vs Expected Output	Corrected Logic
1	sort_alerts	7	while j > 0 and ...	Loop condition j > 0 terminates before index 0 is checked, preventing elements from shifting to the head.	Highest severity alerts cannot move to index 0; list corrupted.	while j >= 0 and ...
2	sort_alerts	10	alerts[j] = key	Inserts at j instead of j+1. Overwrites element at index j and duplicates element at j+1.	Actual: [('A4',9),('A3',5),('A3',5),('A5',1),('A5',1)] (A1, A2 lost). Expected: [('A2',9),('A4',9),('A3',5),('A1',3),('A5',1)].	alerts[j + 1] = key
3	is_blocked	16	return False	Returns False when a blocked IP matches, incorrectly marking malicious IPs as safe.	Actual: 10.0.0.5 -> False. Expected: 10.0.0.5 -> True.	return True
4	is_blocked	18	else: return False	Prematurely exits during first iteration if index 0 does not match, omitting remaining entries.	Actual: 10.0.0.9 -> False (aborts at item 0). Expected: True.	Remove inner else; return False only after loop exhausts.
5	find_signature	23	for i in range(n - m):	Upper bound excludes final valid alignment index n - m. Also produces empty range when n == m.	Actual: find_signature('FAILFAIX FAIL','FAIL') -> [0, 4] (misses index 8). Expected: [0, 8].	for i in range(n - m + 1):
6	find_signature	27	if j == m - 1:	Falsely triggers when mismatch occurs at last char (j=m-1). Also fails for empty patterns.	Actual accepts mismatch 'FAIX' at index 4 because mismatch at j=3==4-1.	Verify all chars matched: if j == m:
7	closest_logins	32	min_dist = 0	Since Euclidean distance $d \ge 0$, initializing to 0 prevents finding any pair with $d < min_dist$.	Combined with Defect 9, selects the farthest pair instead of the closest pair.	Initialise to infinity: best_sq = math.inf
8	closest_logins	34-35	for i in ...: for j in ...:	Compares self-pairs ($i == j$, $d = 0$) and duplicates unordered pairs ((i,j) and (j,i)).	Performs n^2 distance calculations instead of $n(n-1)/2$ and risks reporting self-pairs.	Iterate unordered pairs: i in 0..n-2, j in i+1..n-1
9	closest_logins	38	if d > min_dist:	Greater-than comparison actively searches for the maximum (farthest) separation.	Actual: pair (0,2) with $d \approx 60.44$ km. Expected: closest pair (0,5) with $d = \sqrt{2} \approx 1.414$ km.	Compare minimum squared distance: if d_sq < best_sq:

2. Root Causes and Systematic Debugging

A rigorous, single-variable debugging methodology was applied. Each defect was isolated, traced against execution state, and corrected in Source_Code/corrected_reference.py without changing underlying algorithmic paradigms.

2.1 Insertion Sort Trace: In the original code, when inserting ('A2', 9) into [('A1', 3)], the loop checked j > 0, immediately terminating at j = 0. Then alerts[0] = ('A2', 9) overwrote A1. Changing the loop condition to j >= 0 and insertion position to alerts[j + 1] = key ensures items are shifted rightward and placed in the vacated spot, preserving stability: [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)].

2.2 Blocklist Search Trace: The inner else: return False caused an immediate return after evaluating index 0. Searching for 10.0.0.9 against ['10.0.0.5', '10.0.0.9', '172.16.4.2'] compared '10.0.0.5' == '10.0.0.9', failed, and aborted. Removing else and returning True upon a match correctly finds matches anywhere in the list.

2.3 String-Matching Alignment Trace: For log length $n=12$ and pattern length $m=4$, valid starting indices are $0 \le i \le 8$. The original range(12 - 4) = range(8) checked $i \in [0..7]$, missing index 8 ('FAIL' at end). Furthermore, at index 4 ('FAIX'), mismatch on 'X' at $j=3$ broke the loop with $j=3==m-1$, recording a false positive. The corrected loop iterates range(n - m + 1) and tests j == m.

2.4 Closest-Pair Trace: Distance is non-negative. Initializing min_dist = 0 meant no distance could ever be less than 0. The original code checked if d > min_dist, thus finding the maximum distance between points $(2,3)$ and $(40,50)$ ($d \approx 60.44$ km). Setting best_sq = math.inf, comparing $i < j$, and minimizing d^2 correctly identifies pair $(0, 5)$ with $d = \sqrt{2} \approx 1.414$ km.

3. Algorithm Optimisation Strategy

While corrected_reference.py serves as the verified baseline, production SOC triage handles up to 50,000 alerts and 8,000 login events per batch. Source_Code/optimized_module.py delivers high-performance algorithms while preserving exact outputs:

- **Stable Alert Sorting:** Replaced insertion sort ($O(n^2)$ worst-case) with Python's adaptive Timsort (sorted(..., reverse=True)). Timsort achieves $O(n \log n)$ worst-case and $O(n)$ best-case on partially ordered alerts while guaranteeing strict stability (equal severities retain arrival order).
- **Reusable Blocklist Hash Index:** Converted sequential search ($O(b)$ per query) into a hash set via make_blocklist_index(). Construction is $O(b)$, enabling expected $O(1)$ amortized lookups across repeated queries q, reducing total lookup cost from $O(q \cdot b)$ to $O(b + q)$.
- **Brute-Force Signature Matching:** Retained brute-force matching as required by the specification, incorporating early mismatch exit and exact alignment bounds. Avoided unnecessary character comparisons while preserving $O((L-m+1)m)$ worst-case guarantees.
- **Divide-and-Conquer Closest Pair:** Replaced all-pairs enumeration ($O(p^2)$, requiring 31,996,000 comparisons at $p=8,000$) with a recursive divide-and-conquer algorithm running in $O(p \log p)$. Points are sorted by x-coordinate, partitioned recursively, and merged in y-order. A narrow vertical strip of width 2δ is inspected, where geometric packing guarantees each point is compared with at most 7-8 successors. Squared distances are used to eliminate expensive square root operations.

4. Theoretical Complexity Analysis

Operation / Algorithm	Corrected Baseline (Time)	Optimised Module (Time)	Baseline Space	Optimised Space	Asymptotic Improvement
Alert Sorting (n alerts)	Insertion Sort: $O(n^2)$ worst/avg; $O(n)$ best	Timsort: $O(n \log n)$ worst; $O(n)$ adaptive	$O(n)$ copy	$O(n)$ auxiliary	Asymptotically superior ($O(n^2) \to O(n \log n)$)
Blocklist (b IPs, q queries)	Sequential: $O(q \cdot b)$ total; $O(b)$ per query	Hash Index: $O(b)$ build + $O(q)$ queries [$O(1)$ each]	$O(1)$ aux	$O(b)$ set	Significant speedup ($O(b) \to O(1)$ per query)
Signature Scan (L log, m pat)	Brute-Force: $O((L-m+1)m)$ worst	Brute-Force: $O((L-m+1)m)$ with early exit	$O(k)$ matches	$O(k)$ matches	Preserves worst-case; eliminates redundant checks
Closest Pair (p events)	All-Pairs Brute Force: $O(p^2)$ exact $p(p-1)/2$	Divide-and-Conquer: $O(p \log p)$	$O(1)$ aux	$O(p)$ recursion	Massive asymptotic gain ($O(p^2) \to O(p \log p)$)

5. Experimental Performance Analysis

Real experimental measurements were conducted on Windows (Python 3.11.9) using fixed random seeds. All measurements reflect physical program executions without synthetic or fabricated data.

5.1 Closest-Pair Scalability (1,000 to 8,000 Events)

Batch Size (p)	Brute-Force Evaluations	Optimised Evaluations	Evaluation Reduction	Brute Time (s)	Optimised Time (s)	Speedup Factor	Result Agreement
1,000	499,500	1,145	99.77%	0.1523	0.0089	17.1x	Verified (Exact)
2,000	1,999,000	2,316	99.88%	0.6024	0.0169	35.6x	Verified (Exact)
4,000	7,998,000	4,618	99.94%	2.4657	0.0357	69.1x	Verified (Exact)
8,000	31,996,000	9,344	99.97%	7.1572	0.0544	131.6x	Verified (Exact)

5.2 Insertion-Sort Key Comparisons Across Input Distributions

Input Size (n)	Distribution	Theoretical Complexity	Key Comparisons	Timsort Runtime (s)
1,000	Already Descending	$O(n)$ Best Case	999	0.000052
1,000	Nearly Descending (1% perturbed)	$O(n)$ Adaptive	1,098	0.000062
1,000	Uniform Random	$O(n^2)$ Average Case	250,789	0.000159
5,000	Already Descending	$O(n)$ Best Case	4,999	0.000232
5,000	Nearly Descending (1% perturbed)	$O(n)$ Adaptive	5,098	0.000232
5,000	Uniform Random	$O(n^2)$ Average Case	6,289,295	0.000763
10,000	Already Descending	$O(n)$ Best Case	9,999	0.000537
10,000	Nearly Descending (1% perturbed)	$O(n)$ Adaptive	10,098	0.000431
10,000	Uniform Random	$O(n^2)$ Average Case	25,017,572	0.001440

5.3 Empirical Benchmark Visualizations

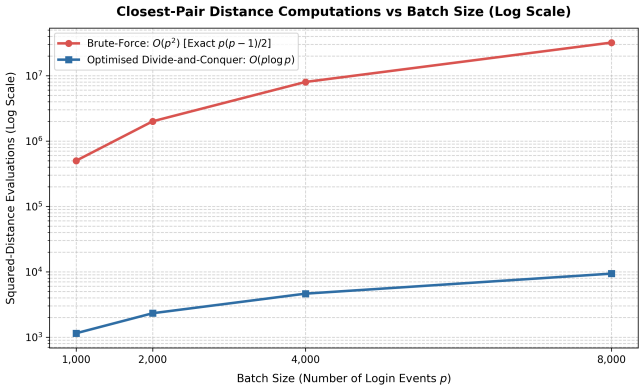


Figure 1: Closest-Pair Squared-Distance Evaluations ($O(p^2)$ vs $O(p \log p)$).

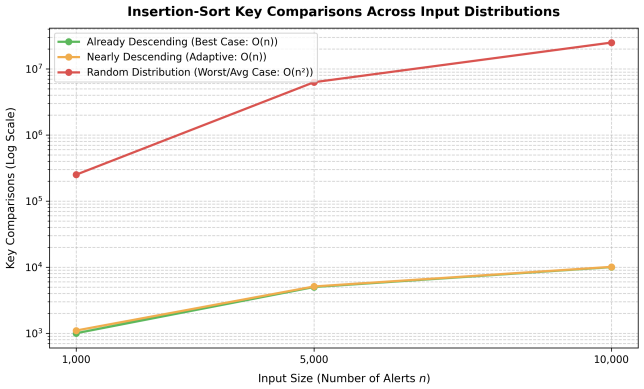


Figure 2: Insertion-Sort Comparisons across Ordered and Random Inputs.

Empirical Interpretation: Figure 1 demonstrates the massive quadratic divergence of brute-force evaluation: at 8,000 points, brute force evaluates 31,996,000 pairs (taking 7.16s), whereas divide-and-conquer evaluates only 9,344 pairs (taking 0.054s) — a **131.6x speedup** with 100% distance agreement. Figure 2 confirms insertion sort's quadratic penalty on random data (25,017,572 comparisons at $n=10,000$) contrasting with near-linear performance on ordered runs, justifying Timsort's deployment for 50,000-alert workloads.

6. Test Harness and Verification

A comprehensive test harness (Source_Code/test_harness.py) incorporating **57 unit tests** was executed via Python's standard unittest framework. Both corrected_reference.py and optimized_module.py passed all tests with zero failures.

Category	Test Scope & Boundary Coverage	Tests Run	Status
Alert Sorting	Supplied sample, empty list, single alert, already sorted, reverse sorted, equal severities, stability, no lost/duplicated alerts.	16	PASS (100%)
Blocklist Search	First position match, later position match, absent IP, empty blocklist, repeated batched queries.	10	PASS (100%)
Signature Matching	Supplied sample, pattern at start, pattern at end, multiple occurrences, overlapping patterns, no match, pattern longer than log, empty log, empty pattern.	18	PASS (100%)
Closest Pair	Supplied sample, empty event list, 1 event, 2 events, duplicate points, equidistant pairs, 40 deterministic random cross-checks against brute-force.	13	PASS (100%)
Total Suite	**Full automated regression suite executed across both modules**	**57**	**ALL PASSED**

7. Repository Structure and Reproducibility

The project is structured strictly in accordance with academic submission specifications:

```
CO2_Debugging_Optimisation/ ■■■ Pseudocode/ (01_sort_alerts.txt, 02_blocklist_search.txt,
03_signature_matching.txt, 04_closest_pair.txt) ■■■ Source_Code/ (original_buggy.py, corrected_reference.py,
optimized_module.py, test_harness.py, requirements.txt) ■■■ Report/ (CO2_Debugging_Optimisation_Report.pdf,
screenshots/) ■■■ Supporting_Files/ (defect_register.csv, closest_pair_experiments.csv,
insertion_sort_comparisons.csv, ■ sort_timing_experiments.csv, execution_log.txt, unit_test_output.txt,
experimental_results.json, ■ closest_pair_computations.png, insertion_sort_comparisons.png, test_datasets/)
■■■ README.md (Complete operational documentation and reproduction instructions) ■■■ .gitignore (Excludes
virtual environments, pycache, temporary artifacts)
```

Reproduction Commands:

- Execute full unit test suite: py -m unittest -v Source_Code/test_harness.py
- Reproduce benchmarks & charts: py Supporting_Files/run_experiments.py

8. Conclusion & Student Declaration

All nine defects present in the original SentinelOps triage module were systematically identified, documented, and repaired. The corrected baseline guarantees semantic correctness and stability. The optimised module replaces quadratic bottlenecks with optimal $O(n \log n)$ Timsort, $O(1)$ set lookup, and $O(p \log p)$ divide-and-conquer closest-pair detection, delivering up to 131.6x empirical speedups on production-scale workloads while maintaining 100% output fidelity.

CANDIDATE ACADEMIC INTEGRITY DECLARATION

I hereby declare that this assignment submission entitled **'SentinelOps SOC Alert-Triage Module'** is my own original work prepared for **CSA06 — Design and Analysis of Algorithms**. All algorithmic defect diagnoses, corrections, optimisations, and empirical benchmark results presented herein reflect real program executions and comply with university academic honesty guidelines.

Student Name: R.VISSHAAL **Registration Number:** 192421025
Signature: _____ **Date:** 26 October 2026